

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

# SC2002 – Restaurant Rush

| **Lab Group** | **SCEE** | **Group 04** |
|---|---|---|
| | | |
| **Members** | Ong Yew Sien, Keane | U2521832D |
| | Lim Ki Rin | U2521121C |
| | Yeo Wei Xuan | U2521952H |
| | Chua Kai Wen | U2520568E |
| | | |
| **Repository and Stage tags** | https://github.com/kyriennn/SC2002-Restaurant-Rush-Game | | Stage-1-release | Stage-2-release | Stage-3-release |

# 1. Project overview and specification

## 1.1 Game concept and manager decision

**Game Concept:** Restaurant Rush is a turn-based restaurant simulation where the player manages a Waiter and a Chef over 18 turns. Customers arrive on turns 1, 4, 7, 10, 13, and 16, join a queue, are seated at one of two tables, place an order, wait for the Chef to prepare it, then eat and pay. Customers lose satisfaction while waiting or sitting unserved and leave unhappy if it reaches zero, so the player must balance seating, ordering, cooking, and serving with limited turns. The game ends in Victory if, by turn 18, enough customers have been served, revenue targets are met, and average satisfaction exceeds a threshold. Three customer types (Regular, VIP, and Critic) differ in patience and pricing, while Stage 3 adds a Host, combo meals, and a one-time Happy Hour to make the game more dynamic.

**Manager Decision:** The Waiter can act once per turn, so when both serving a ready dish and seating a waiting customer are possible, the manager must know how to balance and prioritise. Serving first immediately stops satisfaction loss for the seated customer and moves them into the eating-and-payment pipeline quickly, helping tables become available. Seating first leaves a completed order waiting, allows satisfaction to continue falling, and can create a backlog that increases the risk of unhappy departures. This highlights that the Waiter's single action directly influences customer satisfaction, revenue, and overall success.

## 1.2 Reproducible base setting

| **Item** | **Required value / report evidence** |
|---|---|
| Turns and capacity | 18 turns; 2 tables; game runs all 18 turns, with no early ending. |
| Staff | 1 Waiter: at most one task/turn.<br>1 Chef: one preparation unit/turn.<br>1 Cashier: at most one automatic payment/turn. |
| Arrivals | One customer at the start of turns 1, 4, 7, 10, 13 and 16; six total. |
| Victory at end of turn 18 | At least 4 paid customers with revenue at least $45.00 and an average satisfaction of paid customers at least 60; otherwise Defeat. |
| Menu | Salad $9.00 / 1 unit; Burger $15.00 / 1 unit; Pasta $18.00 / 2 units. |

## 1.3 Global rules and information shown

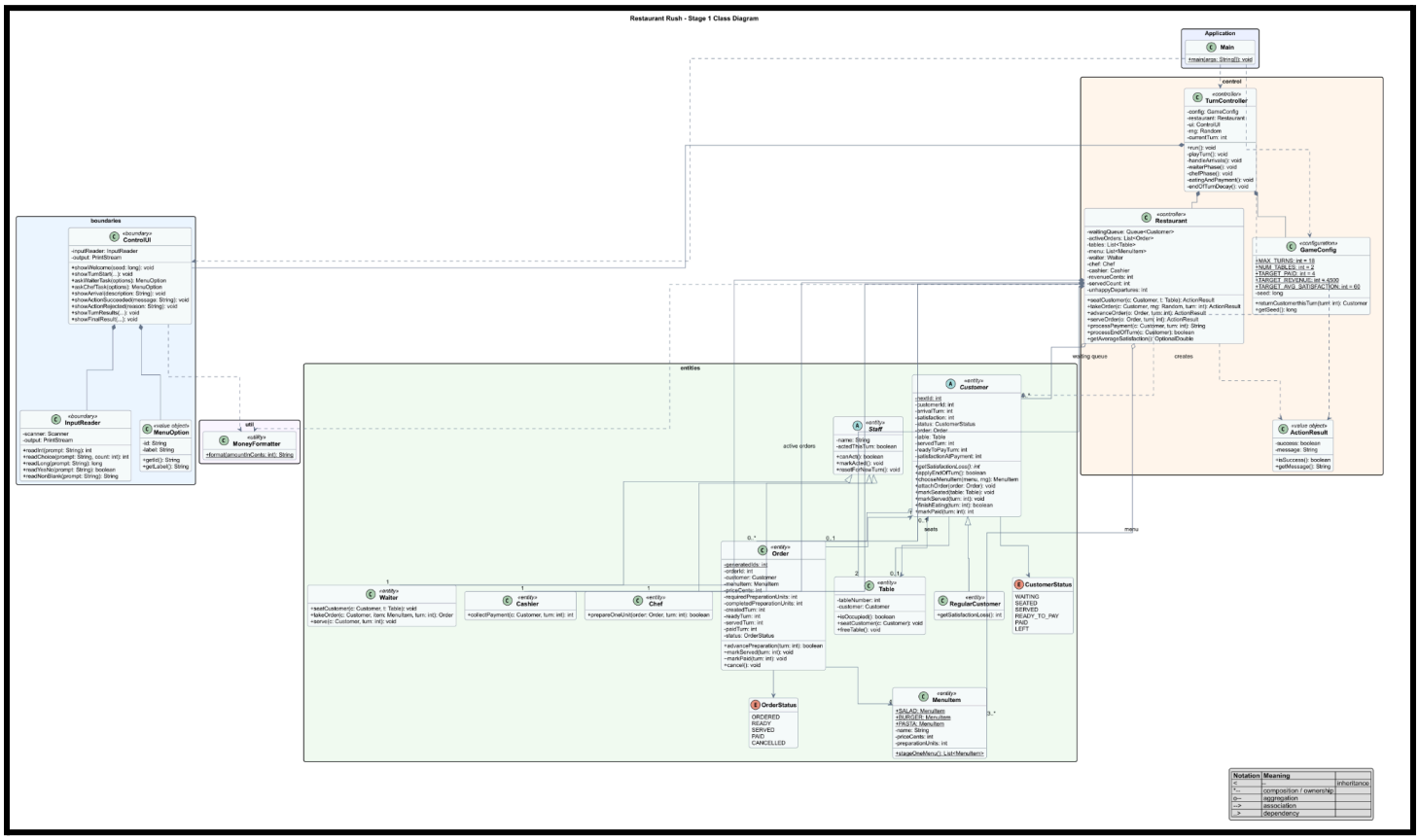
- Customers choose their own food by a documented deterministic rule or seed: 42
- Command-Line Interface (CLI) collects decisions and displays state; domain/control objects own game rules. Mandatory features work offline. Store fractional money in integer cents and display dollars to two decimals.
- At each turn start, show turn number, revenue, average paid-customer satisfaction, waiting queue, occupied tables, each customer's satisfaction and order status, and available staff tasks.
- At the end of turn, show served count, unhappy departures, revenue, average satisfaction of paid customers and turns used. Record satisfaction at payment; show N/A if no payment has occurred.

## 1.4 Turn sequence and invariants

| **Order** | **Required behaviour / evidence to insert** |
|---|---|
| 1 Arrivals | The scheduled customer enters the waiting queue at satisfaction 100. |
| 2 Choices | Waiter chooses one of: seat one queued customer at a free table; take one seated customer order; serve one READY order; or Wait. The chef chooses one order for one preparation unit or Wait. |
| 3 Work | The waiter acts before the Chef. The chef may advance an order taken this turn. The last unit makes an order READY; a dish finished this turn cannot be served until a later turn. |
| 4 Eating/payment | A customer served earlier spends one full turn eating, then becomes READY_TO_PAY. Cashier pays the oldest eligible customer, at most one per turn; newly READY_TO_PAY pays no earlier than next turn. Payment records bill/satisfaction and frees table. |
| 5 Loss/departure | At turn end, waiting or seated, unserved customers lose satisfaction to minimum 0 (Regular 8; see Stage 2). At 0 they leave unpaid; cancel unfinished order and free table. Served/eating/ready-to-pay customers lose nothing. |
| 6 Result | After the full 18 turns, check all three victory targets together. |

# 2. Stage 1: Restaurant Rush Baseline

## 2.1 Stage 1 Class Diagram




*Stage 1 Class Diagram*

## 2.2 Baseline responsibilities and choices

| **Class /component** | **Responsibility and code location** |
|---|---|
| TurnController | Runs the 18-turn game loop, handles arrivals and turn phases, creates the available Waiter and Chef choices, executes the selected actions, processes eating and payment, applies satisfaction loss, and checks the victory conditions.<br>● Location: src/control/TurnController.java |
| Restaurant | Maintains the waiting queue, tables, menu, active orders, staff and results counters. It validates and performs seating, order-taking, food preparation, serving, payment and customer-departure operations.<br>● Location: src/control/Restaurant.java |
| ControlUI | Display the restaurant state, available staff actions, events and final result. It receives and validates the player's Waiter and Chef selections without directly changing the game state.<br>● Location: src/boundaries/ControlUI.java |

- During each turn, TurnController creates the actions that are valid for the current restaurant state. For the Waiter, the player chooses which waiting customer to seat at which free table, which seated customer's order to take, or which ready order to serve.
- The player determines the ordering priority but does not choose the food, because the customer selects their menu item when Restaurant.takeOrder() is called.
- For the Chef, the player chooses which ORDERED order receives one preparation unit. Each staff phase ends after one successful action, so the Waiter and Chef can each complete only one task per turn. This creates constrained-staff decisions.
- For example, when a ready dish and a waiting customer both require the Waiter, the player must choose between serving the dish to stop the seated customer from losing more satisfaction or seating the queued customer to reduce their waiting time. Similarly, if several orders are being prepared, the Chef can advance only one selected order during that turn.

## 2.3 The transition between stage 1 and stage 2

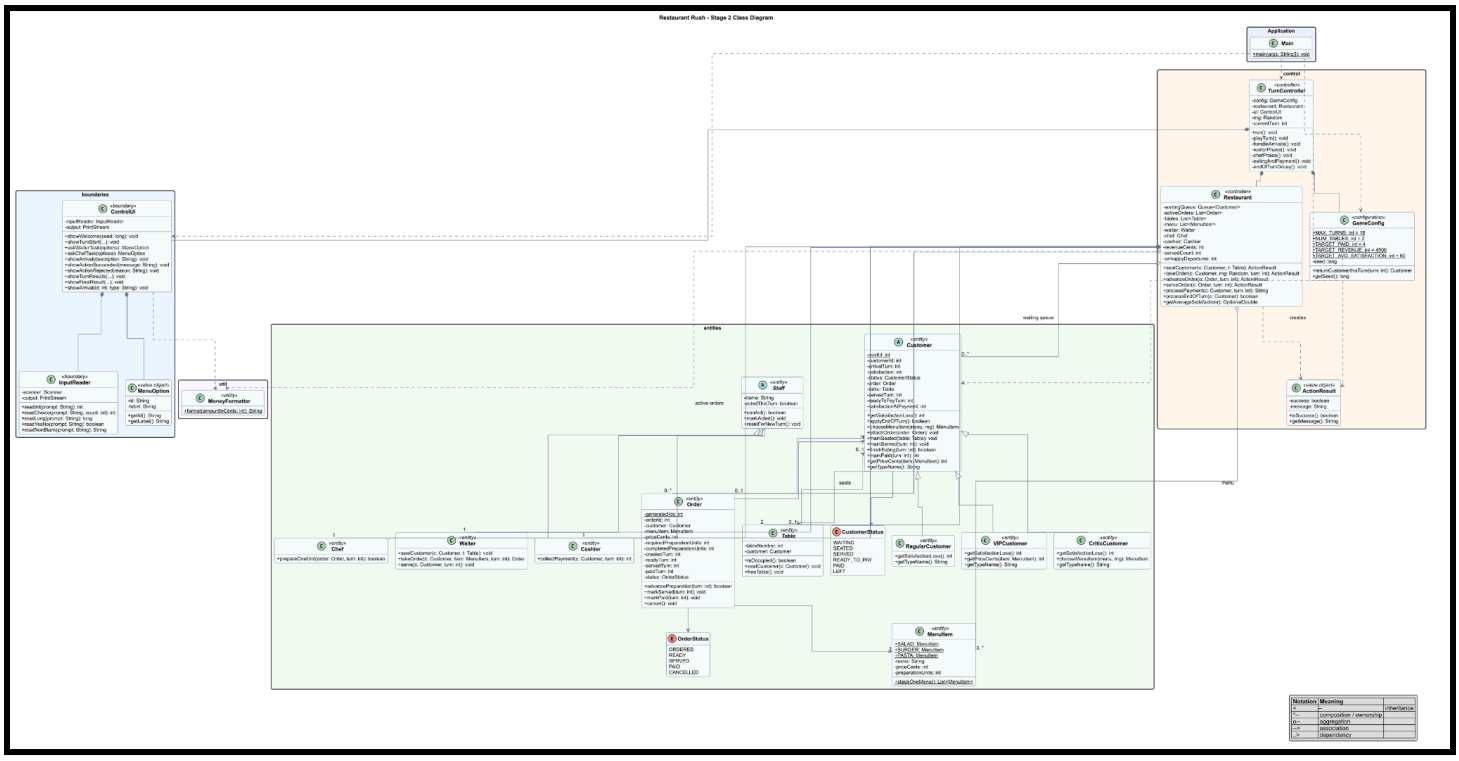
Stage 2 extends the Stage 1 customer hierarchy by introducing VIPCustomer and CriticCustomer. A VIP customer loses 5 satisfaction points during each applicable end-of-turn decay and receives a 10% price discount. A Critic customer loses 12 satisfaction points, pay the normal price and always chooses the highest-priced menu item. Regular customers retain the Stage 1 behaviour of losing 8 satisfaction points and paying the normal price. These differences are implemented polymorphically through overridden Customer methods instead of customer-type checks in the controller.

| **Evidence** | **Fill in** |
|---|---|
| Compare/commits | **Stage comparison:** stage-1 → stage-2, from commit 9133246 to eae1e2c. The transition changed 10 files with 330 insertions and 22 deletions.<br>**Key commits:** Add VIP/Critic classes and polymorphic pricing; add Stage 2 UI displays; configure customer types and arrivals; display customer types in the controller; and add the Critic's special menu choice. |
| New classes/files | src/entities/VIPCustomer.java: new Customer subtype that loses 5 satisfaction points per applicable turn, applies a 10% discount through getPriceCents() and identifies itself as "VIP". src/entities/CriticCustomer.java new Customer subtype that loses 12 satisfaction points, pays the inherited normal price, identifies itself as "Critic" and overrides chooseMenuItem() to select the highest-priced item. |
| Changed/deleted classes/files | - Customer.java added the default full-price<br>- getPriceCents(MenuItem) method and abstract getTypeName() method to support polymorphism.<br>- Order.java obtains and locks the price through customer.getPriceCents(menuItem), allowing VIP discounts without type checks.<br>- RegularCustomer.java implements getTypeName() while retaining its 8-point satisfaction loss.<br>- GameConfig.java retains the six arrival turns but creates a VIP on turn 4 and a Critic on turn 10.<br>- TurnController.javaadds customer-type information to arrival, seating, queue and status displays; its turn phases remain unchanged. |

| | |
|---|---|
| | - ControlUI.java adds type-aware labels and methods for displaying normal and discounted prices, customer behaviour and payments.<br>- README.md and Restaurant_Rush_Blueprint.md were updated with Stage 2 rules and example strategies. No classes or files were deleted. |
| Earlier behaviour retained | - Stage 2 compiled successfully and a complete seed-42 regression run confirmed that the original 18-turn loop, two tables, six arrival turns, Salad/Burger/Pasta menu, Waiter/Chef/Cashier workflow, order lifecycle, satisfaction abandonment, automatic payment and victory thresholds still operate.<br>- Restaurant.java, Waiter.java, Chef.java, Cashier.java, Staff.java, Table.java, MenuItem.java and the status enums remain unchanged.<br>- The regression run completed the full seat → order → prepare → READY → serve → eat → pay lifecycle and ended in **VICTORY** with 4 served, 4 paid, 0 unhappy departures, $53.10 revenue and 72.25/100 average satisfaction after 18 turns. |

# 3. Stage 2 on customer behaviour

## 3.1 Stage 2 Class Diagram




*Stage 2 Class Diagram*

## 3.2 Price and Distinct Behaviour

| Customer | Loss / unserved turn | Price and distinct behaviour |
|---|---|---|
| Regular | 8 | Pays full price and selects a menu item randomly using the seeded random generator. |
| VIP | 5 | Receives a 10% discount and is charged 90% of the menu price. It otherwise uses the normal seeded menu selection. |
| Critic | 12 | Pays full price and always selects the highest-priced menu item. In the Stage 2 menu, this is Pasta at $18.00. |

Stage 2 retains six customer arrivals at turns 1, 4, 7, 10, 13 and 16. The schedule is: Regular on turn 1, VIP on turn 4, Regular on turn 7, Critic on turn 10, and Regular customers on turns 13 and 16. The game continues to use the fixed seed 42, which makes the Regular and VIP menu selections reproducible.

All three customer types inherit from the abstract Customer class and follow the same lifecycle. WAITING → SEATED → ORDERED → READY → SERVED → READY_TO_PAY → PAID.

The same Restaurant operations handle seating, ordering, preparation, serving and payment regardless of customer type. Their differences are dispatched polymorphically through customer methods: getSatisfactionLoss() determines satisfaction reduction, getPriceCents() determines the charged price, and chooseMenuItem() determines menu preference. Therefore, the controller does not require instanceof checks or separate service flows for VIP and Critic customers.

The Stage 1 victory and defeat criteria are retained. After 18 turns, victory still requires at least 4 paid customers, revenue of at least $45.00, and an average paid-customer satisfaction of at least 60/100. If any of these three conditions is not met, the result is defeat.

## 3.3 The transition between stage 2 and stage 3

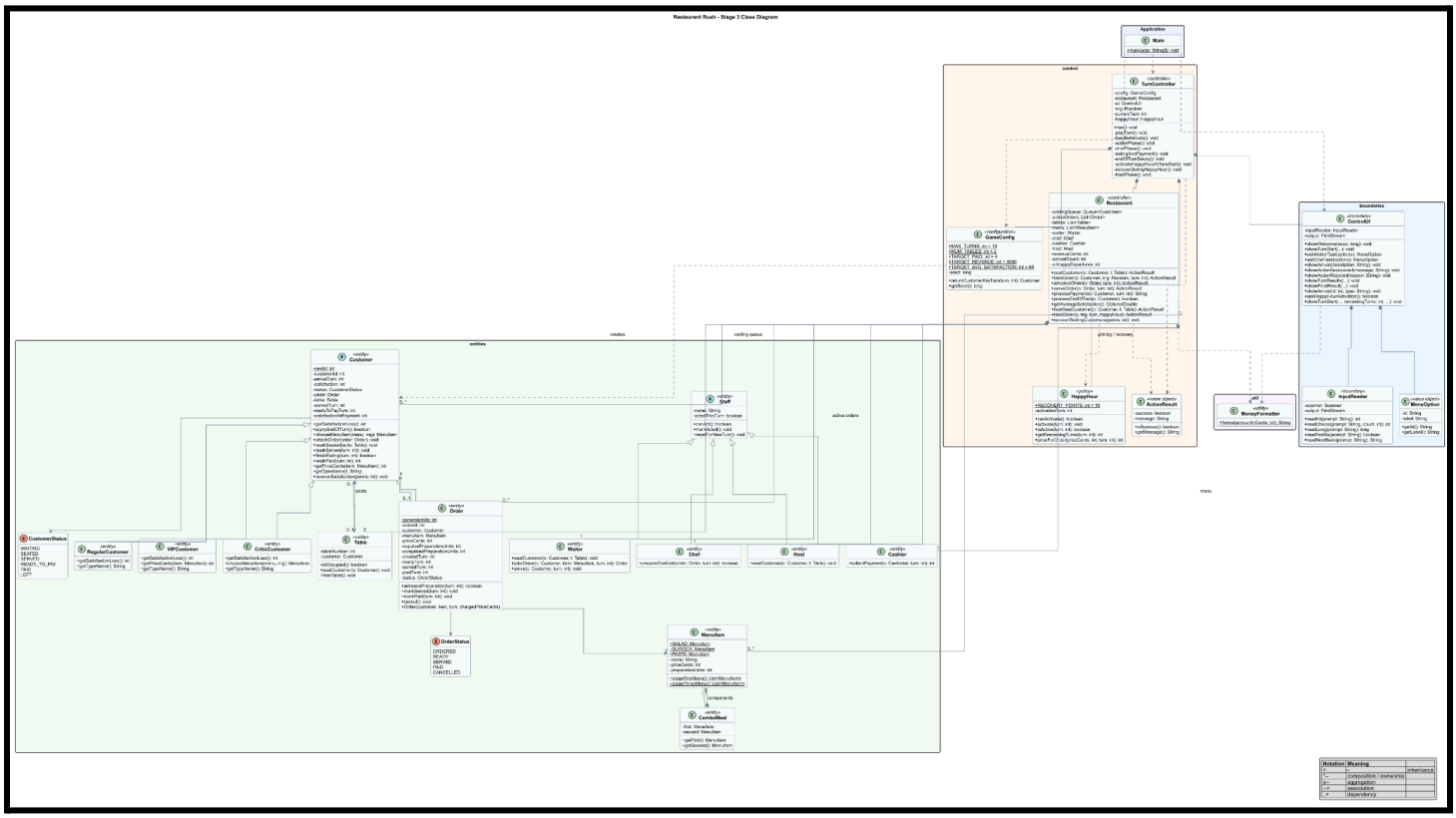
Stage 3 extends Stage 2 with a new Host role, composed menu items and a temporary pricing policy. The Host automatically seats the first queued customer at the first available table before the Waiter phase, at most once per turn. The Waiter retains the ability to seat customers independently. A ComboMeal combines exactly two ordinary menu items, requires the sum of their preparation units and costs 90% of their combined prices. HappyHour can be activated once per game and remains active during the activation turn and the following turn. During these turns, waiting and unserved customers recover 15 satisfaction points, and newly placed orders receive a 20% discount after any customer-specific discount. The final price is locked when the order is created.

| **Evidence** | **Fill in** |
|---|---|
| Compare/commits | The repository does not contain a stage-3 tag, so Stage 3 is represented by main commit c819526. **Comparison:** stage-2 → Stage 3 snapshot. The complete transition changed 15 files with 654 insertions and 36 deletions. **Key commits:** add Host, ComboMeal and HappyHour; integrate Stage 3 into the controller, restaurant and entities; HappyHour and Critic documentation follow-up; Stage 3 blueprint; and sequence-diagram source. |
| New classes/files | - src/entities/Host.java: new Staff subtype that seats at most one queued customer per turn independently of the Waiter.<br>- src/entities/ComboMeal.java: new MenuItem subtype composed of two ordinary menu items; its preparation units are summed and its price is 90% of the combined component prices. Nested combos are rejected.<br>- src/control/HappyHour.java: encapsulates one-time activation, its two-turn duration, remaining turns, 15-point satisfaction recovery and the 20% promotional discount. docs/diagrams/sequence-diagram.puml and sequence-diagram.png were added as design-documentation files. |
| Changed/deleted classes/files | - MenuItem.java removed final so ComboMeal can inherit from it; retained the Stage 1 menu and added a Stage 3 menu containing the original items and three pairwise combos.<br>- Restaurant.java now owns the Host, uses the Stage 3 menu, performs Host seating, composes customer and Happy Hour discounts, locks the final order price and recovers eligible customers.<br>- TurnController.java adds Happy Hour activation/recovery and the automatic Host phase before the Waiter phase.<br>- ControlUI.java adds the Happy Hour activation prompt and remaining-turn display while retaining the earlier display overload.<br>- Customer.java adds satisfaction recovery capped at 100 for WAITING and SEATED customers.<br>- Order.java adds constructors that accept an already-calculated charged price so it remains locked after ordering.<br>- GameConfig.java adds the 15-point Happy Hour recovery constant while retaining the original arrivals and victory targets.<br>- CriticCustomer.java only adds a comment clarifying that price ties use menu order; its behaviour is unchanged.<br>- README.md and Restaurant_Rush_Blueprint.md were updated with Stage 3 rules and test evidence. No classes or files were deleted. |
| Earlier behaviour retained | - Stage 3 compiled successfully. The game still uses 18 turns, two tables, six arrivals on turns 1, 4, 7, 10, 13 and 16, and the same Regular/VIP/Critic schedule. Regular customers still lose 8 satisfaction points, VIP customers lose 5 and receive their 10% discount, and Critics lose 12 and choose the highest-priced item. The Salad, Burger and Pasta items remain available. |

| | |
|---|---|
| | - The original seat → order → prepare → READY → serve → eat → pay lifecycle and victory requirements of 4 paid customers, $45.00 revenue and 60/100 average satisfaction are retained. A verified seed-42 victory run ended with 6 served, 5 paid, 0 unhappy departures, $90.25 revenue and 82.20/100 average satisfaction. A second run with no Happy Hour and the Waiter always waiting ended in DEFEAT with 0 served, 0 paid, 2 unhappy departures and $0.00 revenue. |
| Design effect | Stage 3 is an additive extension rather than a broad rewrite. Host extends the existing Staff abstraction, separating automatic seating from the Waiter's limited action while preserving the Waiter's seating capability. ComboMeal extends and composes MenuItem, allowing the existing customer, order and kitchen logic to treat individual dishes and combos uniformly. Consequently, the Critic's existing highest-price rule automatically considers combos without controller changes. HappyHour isolates time, recovery and promotional-pricing rules in a dedicated policy class. Pricing now follows a clear sequence: menu or combo price → customer-specific discount → Happy Hour discount → locked Order price. Earlier constructors, method overloads and stageOneMenu() were retained for compatibility, keeping UI, orchestration and domain-policy responsibilities separated. |

# 4. Stage 3: role and menu policy

## 4.1 Stage 3 class diagram




*Stage 3 Class Diagram*

## 4.2 Stage 3 rules and evidence

| Feature | Required rule and what to demonstrate |
|---|---|
| Host | The Host automatically seats the first queued customer at the first free table before the Waiter phase, at most once per turn. The Waiter retains a separate action and can still seat another customer. **Demonstration run:** The Waiter waited during turns 1–8, took Customer 2's order on turn 9, completed preparation on turns 9–10 and served Customer 2 on turn 11. At the end of turn 13, Customer 2 paid and Customer 1 left due to zero satisfaction, leaving both tables free. On turn 14, the Host automatically seated Customer 3 at Table 1. During the same turn, the Waiter selected "Seat Customer 4 at Table 2", which succeeded. This demonstrates the Host and Waiter both seating customers during one turn. |
| ComboMeal | A ComboMeal contains exactly two ordinary MenuItem objects. Its preparation units are the sum of its two components, while its price is 90% of their combined price, rounded to the nearest cent. The Stage 3 menu contains Salad + Burger for $21.60 and 2 units, Salad + Pasta for $24.30 and 3 units, and Burger + Pasta for $29.70 and 3 units. Nested combos are rejected. Regular and VIP customers select randomly from the complete six-item menu using seed 42. Critics select the highest-priced item, which is the Burger + Pasta Combo. The manager does not choose the customer's food. |
| Happy Hour activation | The manager may activate Happy Hour once per game at the beginning of a turn. It remains active for the activation turn and the following turn. In the verified victory run, the manager answered N before turns 1–3 and Y before turn 4. The interface displayed 2 remaining turns on turn 4, 1 remaining turn on turn 5 and 0 remaining turns on turn 6. The game did not ask for another activation after it had been used. |
| Happy Hour effects | Orders placed while Happy Hour is active receive a 20% discount. At the start of each active turn, customers with status WAITING or SEATED recover 15 satisfaction points, capped at 100. Customers who have left, are eating, are ready to pay or have already paid are not affected. In the verified run, the turn-4 VIP customer recovered satisfaction during the active period and received the Happy Hour discount on the order placed during turn 4. Existing orders placed before activation are not repriced. |
| Discount order | Pricing is applied in this order: menu or combo price, then the customer-specific discount, then the Happy Hour discount. The resulting amount is stored in the Order when it is created and remains unchanged. **Calculation check 1:** VIP Burger: 1500 × 0.90 = 1350 cents, followed by 1350 × 0.80 = 1080 cents, giving $10.80. **Rounding check:** Salad + Burger Combo base price is $21.60; VIP discount gives $19.44; Happy Hour gives $15.552, rounded to $15.55. In the verified run, this order was created on turn 4 for $15.55 and paid on turn 8 for the same $15.55, after Happy Hour had expired, demonstrating price locking. |

## 4.3 Design Scope

- **Additional feature:** Instead of registering only one required combo, Stage 3 provides all three pairwise combinations of Salad, Burger and Pasta. ComboMeal also rejects null components and nested combos. This strengthens the composition design and demonstrates that new menu products can be added through MenuItem without replacing the required preparation or pricing rules.
- **Meaningful changes to earlier classes:** MenuItem is no longer final, allowing ComboMeal to extend it, and it now provides a Stage 3 menu containing the three original items and three combos. Restaurant owns the Host, applies the pricing sequence, stores the final locked order price and performs satisfaction recovery. TurnController adds the Happy Hour and Host phases while retaining the original Waiter, Chef, eating, payment and decay phases. Customer adds capped satisfaction recovery. Order adds constructors accepting an already-calculated price. ControlUI adds the Happy Hour prompt and remaining-duration display. GameConfig adds the recovery constant while retaining the original arrivals and victory targets.
- **Rewrite justification:** Stage 3 does not broadly rewrite the stable Stage 1 and Stage 2 behaviour. Earlier constructors, takeOrder() and showTurnStart() overloads, and stageOneMenu() remain available for compatibility. The original customer hierarchy, staff workflow, order lifecycle, satisfaction loss and victory calculation remain in place. Stage 3 introduces its features through inheritance, composition and a separate policy object rather than replacing the existing rules.

# 5. Sequence Diagram

The sequence diagram below shows the Stage 2 use case of serving a VIP customer from arrival to payment. It is split across two figures by turn for ease of viewing. The turn loop continues from figure 5a to figure 5b, while other customers are not shown for clarity.

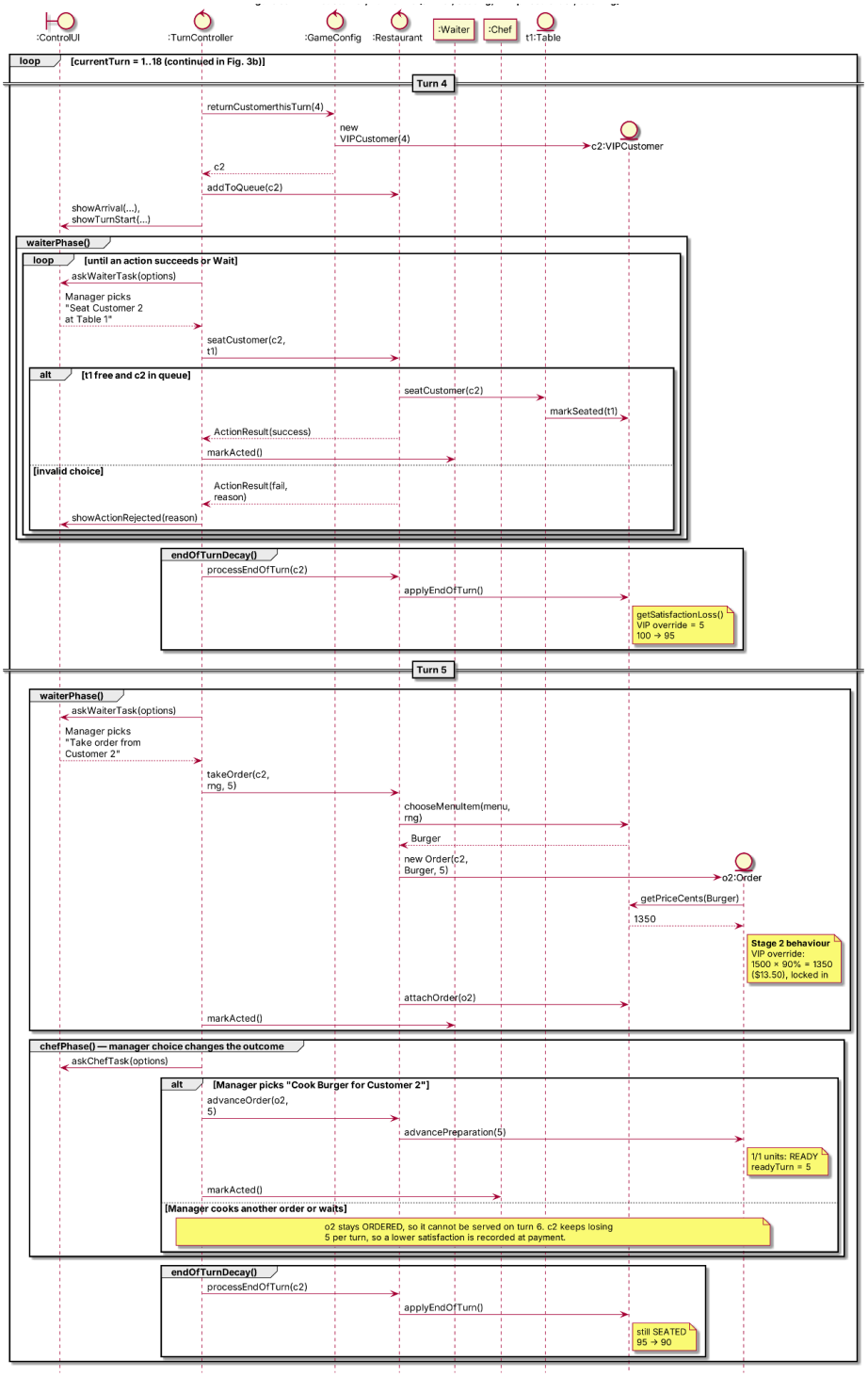



**Figure 5a. Turns 4-5: VIP customer arrives and is seated, orders a Burger item at the VIP price ($13.50 via getPriceCents overridden method), and manager's Chef choice decides whether the dish is ready by turn 5.**

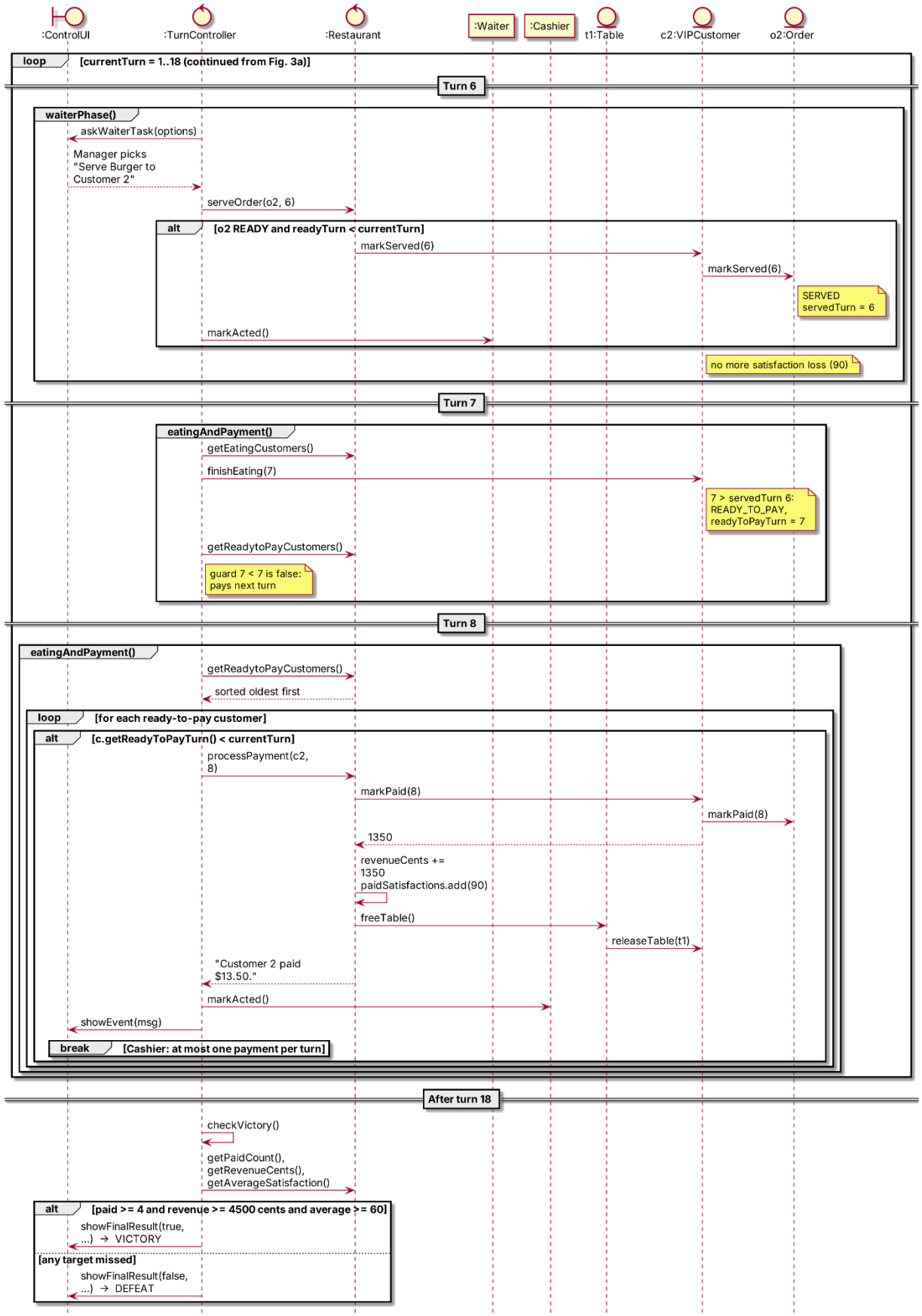



**Figure 5b. Turns 6-8: Burger item is served, VIP customer eats for one turn, and Cashier processes payment (revenue +$13.50, satisfaction 90 recorded, table freed), followed by the end-of-game Victory Check.**

# 6. Design Discussion

## 6.1 Key abstractions and responsibilities

| **Abstraction** | **Responsibility / collaborators / code** |
|---|---|
| Domain entity | Owns state and invariants.<br>Customer: satisfaction, status, order, table; state machine blocks invalid transitions (cannot pay before eating). Customer.java<br>Order: prep progress and state transitions; prepares only when ORDERED, serves only after READY. Order.java |
| Staff role | [One-turn task with constraints.<br>Staff: canAct(), markActed(), resetForNewTurn() ensure one action per turn. Staff.java<br>Waiter, Host, Chef, Cashier perform seating, serving, cooking, and payment collection. |
| Controller | Coordinates turn flow without type checks.<br>TurnController.playTurn() runs phases (arrivals → host → waiter → chef → eat/pay → decay) and calls polymorphic methods. TurnController.java |
| CLI boundary | Handles input and output.<br>ControlUI displays state, InputReader validates input. Game rules remain in the domain layer via ActionResult. |
| Policy/composition | Encapsulates pricing and menu composition.<br>HappyHour provides discount/recovery rules. HappyHour.java<br>ComboMeal combines two MenuItems (90% price, combined prep units). ComboMeal.java |

## 6.2 OOP design supported by actual code

| **Concept** | **Specific example and why it helps** |
|---|---|
| Encapsulation | Fields are private; state changes occur only through guarded methods. Order.advancePreparation() requires ORDERED, markServed() requires the order to be ready, and Customer.markPaid() requires READY_TO_PAY. Invalid states cannot occur. |
| Inheritance and polymorphism | The customer is abstract. Regular, VIP, and Critic override getSatisfactionLoss(), getPriceCents(), and chooseMenuItem(). The controller uses a Customer reference without type checks. |
| Composition | ComboMeal extends MenuItem and contains two component items. The restaurant contains staff, tables, orders, and the queue, managing their lifecycle. |
| SRP / OCP / LSP / ISP / DIP | SRP: ControlUI (display), InputReader (input), MoneyFormatter (currency), GameConfig (rules), and HappyHour (discount policy) each have one responsibility.<br>OCP: New customer or menu types can be added through subclasses without changing TurnController or Restaurant.<br>LSP: VIP and Critic can replace Customer without breaking behaviour because they preserve the same contract. |

## 6.3 Trade-off, alternative, and player agency

- Alternative considered: Put customer-type behaviour in TurnController using switch/if-else statements.
- Explain one manager's decision under limited staff time and compare the resulting customer state or final outcome with another valid choice: The Waiter can act only once per turn. On turns where both serving a ready order and seating a waiting customer are possible (turns 4, 7, 10, 14 and 17 in the seed-42 run), the manager must choose.
    - Serve first: customer satisfaction is preserved, the eating/payment pipeline progresses sooner, and tables become available on time.
    - Seat first: a ready order waits longer, satisfaction continues to decline, and table occupancy can create a backlog that may lead to unhappy departures.
- Identify extension points that were present at each actual stage; avoid crediting later-stage features to an earlier tag.
    - Stage 1: abstract Customer and Staff, extensible MenuItem, and GameConfig.returnCustomerThisTurn() acting as a customer factory.
    - Stage 2: chooseMenuItem() and getPriceCents() allow for polymorphism; GameConfig was extended to create VIPCustomer and CriticCustomer.
    - Stage 3: ComboMeal extends MenuItem, Host extends Staff, and HappyHour provides discount/recovery policy behaviour.

# 7. Functional verification and results

**Build command:** javac -d out src/Main.java src/boundaries/*.java src/control/*.java src/entities/*.java src/util/*.java

**Run command:** java -cp out Main

**Seed:** 42, configured in Main.java

## 7.1 Run log and outcome summary

| Stage / case | Command or choices | Evidence link |
|---|---|---|
| 1 Victory | [1,1,1,2,1,1,2,1,1,2,1,1,1,2,1,1,1,1 ] | FINAL RESULTS Served count: 5 Paid customers: 5 Unhappy departures: 0 Revenue: $63.00 Average satisfaction of paid customers: 74.40/100 Turns used: 18 |
| 1 Defeat | [3,3,3,5,5,5,7,7,7,9,9,9,11,9,9,11,9,9] | FINAL RESULTS Served count: 0 Paid customers: 0 Unhappy departures: 2 Revenue: $0.00 Average satisfaction of paid customers: N/A Turns used: 18 |
| 2 Victory | [1,1,1,2,1,1,1,2,1,1,1,1,1,2,1,1,2,1 ] | FINAL RESULTS Served count: 5 Paid customers: 4 Unhappy departures: 0 Revenue: $53.10 Average satisfaction of paid customers: 71.00/100 Turns used: 18 |
| 2 Defeat | [3,3,3,5,5,5,7,7,7,9,9,9,11,9,9,11,9,9] | FINAL RESULTS Served count: 0 Paid customers: 0 Unhappy departures: 2 Revenue: $0.00 Average satisfaction of paid customers: N/A Turns used: 18 |
| 3 Victory | [(N,1),(Y,1),1,1,1,1,1,1,1,1,1,1,2,1,1,2,1,1] | FINAL RESULTS Served count: 6 Paid customers: 5 Unhappy departures: 0 Revenue: $94.14 Average satisfaction of paid customers: 84.40/100 Turns used: 18 |
| 3 Defeat | [(N,2),(N,2),(N,2),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3),(N,3)] | FINAL RESULTS Served count: 0 Paid customers: 0 Unhappy departures: 2 Revenue: $0.00 Average satisfaction of paid customers: N/A Turns used: 18 |

# 8. Individual reflections

[Ong Yew Sien, Keane U2521832D]

| Prompt | Member-specific response |
|---|---|
| Contribution | I was responsible for the boundary layer in Stages 1 and 2. I implemented src/boundaries/InputReader.java, including reusable validation for integers, bounded choices, long values, yes/no answers and non-blank text.<br>I used injectable Scanner and PrintStream dependencies to make input behaviour easier to test. I also implemented the initial ControlUI.java, covering the welcome and turn displays, waiting queue and table information, dynamic Waiter and Chef menus, action feedback, final results and the MenuOption value object.<br>For Stage 2, I extended ControlUI to display Regular, VIP and Critic labels and separate menu and charged prices without calculating discounts in the UI. |
| Knowledge or skill gained | I learned how to design a command-line boundary around stable interfaces instead of coupling input and output directly to game logic. I gained practical experience with constructor injection, defensive input validation, dynamic menus and method overloading for backward compatibility. I also learned to keep money internally as integer cents while displaying user-friendly values such as $18.00, and to coordinate boundary changes with the controller and entity work completed by other members. |
| Course concept applied | I applied Boundary-Control-Entity separation and the Single Responsibility Principle. InputReader parses and validates raw input, while ControlUI presents information and returns the selected MenuOption TurnController determines which actions are available, and the domain classes calculate customer behaviour, prices and victory conditions. For example, the Stage 2 order-display method receives the original and charged prices from the domain and only formats them. It does not contain VIP discount formulas or customer-type checks. This reduced coupling and allowed the customer hierarchy to remain polymorphic. |
| Hardest challenge | The hardest challenge was keeping the interface clear and correct while the available actions and prices changed with the game state. The Waiter and Chef menus contain different options each turn, so hard-coded menu numbers would have been unreliable. Raw amounts such as 1800 cents also initially needed to be displayed as $18.00.<br>Stage 2 added another difficulty because the interface had to distinguish customer types and show discounted prices without duplicating the pricing rules or breaking the Stage 1 method signatures. |

| Prompt | Member-specific response |
| --- | --- |
| Attempts and resolution | I first separated all parsing and retry loops into InputReader, allowing invalid numbers, out-of-range choices and invalid yes/no answers to be rejected and re-prompted safely. In ControlUI, I accepted the controller's current list of MenuOption objects instead of hard-coding actions. I initially implemented cents-to-dollar formatting in ControlUI to correct outputs such as 1800; during integration, the shared formatting logic was moved into MoneyFormatter so all components used the same representation.<br>For Stage 2, I kept the original arrival and order-display methods and added overloaded methods for type-aware labels and base-versus-charged prices. This preserved earlier call sites while supporting the new requirements. I learned that defensive input handling, backward-compatible interfaces and keeping domain calculations outside the UI make staged integration much safer. |

Working on the boundary package showed me that a user interface is more than a collection of print statements. It forms a contract between the player and the controller, so changes to its parameters or behaviour can affect the entire game. By separating input validation from presentation and keeping pricing and customer rules in the domain layer, I was able to extend the interface for Stage 2 without rewriting the Stage 1 interaction flow. This work also improved my understanding of collaboration through Git because my boundary code had to integrate with controller and entity classes developed by other members.

[Lim Ki Rin, U2521121C]

| Prompt | Member-specific response |
|---|---|
| Contribution | I contributed the control classes: GameConfig, Restaurant, TurnController and ActionResult. I also did the sequence diagram for the team. Additional helper functions like getTypeName() were implemented in entity classes by me to help work hand-in-hand with the control classes and show the key concepts of OOP. |
| Knowledge or skill gained | The biggest learning opportunity was figuring out which class owns which responsibility, and how to keep those responsibilities simple to not affect the ease of extending those classes. Before this project I had the impression that it was better to have classes that could do multiple things so that everything runs through 1 system cleanly, and I thought of classes merely at the entity level. My role in building the control classes taught me that the real question to tackle is who is allowed to know what. Classes like Restaurant are responsible for the game state and rules, so it should not be dependent on the UI, something I took time to understand. For example, when I wanted to show my prices in the messages that restaurant methods returned, I initially tried using ControlUI.formatMoney(), but that would make game logic dependent on UI, so I pivoted to suggesting our team create a utility class Money that can help format for both UI and control classes. |
| Course concept applied | I applied the course's concept of polymorphism through dynamic binding when transitioning from stage 1 to stage 2. Because our Customer class is an abstract class with methods that need to be overridden by each subclass, I programmed the TurnController to only call methods with a Customer reference, like applyEndOfTurn() and chooseMenuItem(), so when VIP and Critic customer types entered in stage 2, Java would choose the correct overriding method at runtime so that features like extra satisfaction loss and reduced bills unique to those special customers would be applied without changing anything logically in the TurnController class. Only display changes had to be made. This shows the Open/Closed principle taught in the lectures, where new behaviour is implemented by creating new classes rather than editing existing code. The Liskov Substitution principle is also applied here since any customer subclass can be used wherever a Customer type is expected. |
| Hardest challenge | Getting the turn timing right. The project brief had a lot of end of turn/ start of turn rules that we were expected to implement carefully, such as cooked dishes not being able to be served until the next turn, and a customer who becomes ready to pay can only pay on the next turn. These were logical errors that would compile and go unnoticed easily. In one version of the serveOrder() method, I wrote the ready-turn check the wrong way around, so nothing could be served at all. Another big challenge was working with my groupmates' classes. Some of their methods already updated the customer status, so my extra calls to certain methods would have crashed the game. |

| Prompt | Member-specific response |
|---|---|
| Attempts and resolution | For the timing rules, I recorded turn numbers on objects such as servedTurn and readyToPayTurn, and compared them to the current turn I was on, instead of relying on the order of method calls which was more confusing. To check my logic, I applied the project brief's Pasta example turn by turn with my code before running the full game.<br><br>As for working with my teammates' merged codes, I learned to understand what my teammates' methods were aiming to do and what I was actually calling instead of inferring off the name itself, and to discuss with them after stage 1 to agree on method names and responsibilities. I also communicated with them whenever I needed to add additional methods in their classes to showcase OOP more consistently, so that they would not be caught off guard when seeing changes made to their files.<br><br>Additionally, I took the time to learn about lambdas and other Java features that we would only learn in week 11 and 12 of our lectures, so that I could apply them to simplify codes and fix bugs that I previously didn't understand.<br><br>The main lesson in all of this was that OOP is only as good as the responsibilities of each class, keeping it simple where one class does one job makes it easier to detect bugs and fix things via the chain of command. |

[Yeo Wei Xuan, U2521952H]

| Prompt | Member-specific response |
|---|---|
| Contribution | Stage 1: I implemented the Customer abstract base with satisfaction/state machine, such as Order, Staff, Table, enums for status<br>Stage 2: I implemented the Critic customer class, which overrides getSatisfactionLoss(), getPriceCents() (VIP 10% off), chooseMenuItem().<br>Stage 3: ComboMeal extends MenuItem, which is the composition of 2 items as a combo, then it will cost 90% of the original price.<br>Help with the ControlUI such that instead of displaying "900 cents", it displays "$18.00" through the use of MoneyFormatter so that the ControlUI does not violate S in SOLID |
| Knowledge or skill gained | Through this project, I strengthened my understanding of Java and object-oriented programming by applying abstraction, inheritance, polymorphism, and encapsulation. I used abstract classes and subclass overriding to model customers and staff, while enums enforced state transitions. I distinguished inheritance from composition in the system design and gained practical experience with Java collections such as Queue, LinkedHashMap, and OptionalDouble. I also improved my ability to separate concerns across application layers, create reproducible program behaviour using fixed seeds, and collaborate effectively through GitHub forking,pulling and status workflows with teammates. |

| Prompt | Member-specific response |
| --- | --- |
| Course concept applied | I applied key object-oriented design principles throughout this project. Inheritance and abstraction were used in the Customer, Staff, and MenuItem hierarchies, while polymorphism allowed customer-specific behaviour without type checking. Encapsulation was enforced through private fields and validated state transitions. Composition was used to build the Restaurant and ComboMeal relationships. I also followed the Single Responsibility Principle by separating UI, input handling, business logic, and utilities into dedicated classes. Additionally, I implemented state-based behaviour with enums, used factory-like object creation, applied defensive programming, and maintained a clear separation between the user interface and domain logic. |
| Hardest challenge | One of the biggest challenges was designing and verifying the pricing and menu-selection system. Customer-specific discounts, Happy Hour discounts, and combo pricing all had to be applied in the correct order with accurate cent-level rounding. Prices also needed to be fixed when an order was placed so later events could not alter them. Additionally, Stage 3 introduced combo meals and required the Critic customer to always select the most expensive menu item, while other customers made deterministic random choices. Ensuring this logic was correct was critical because even small pricing or rounding errors could affect the game's revenue targets and cause reproducible winning sequences to fail. |
| Attempts and resolution | I initially applied discounts in the wrong order and used incorrect rounding for combo pricing, leading to revenue mismatches. I fixed this by applying customer discounts before Happy Hour discounts, locking prices when orders are created, and implementing correct round-half-up calculations, allowing the game's deterministic test run to reproduce the expected results exactly. |

[Chua Kai Wen, U2520568E]

| Prompt | Member-specific response |
|---|---|
| Contribution | In Stage 1, I contributed to the implementation of about half of the game's entity classes to get the baseline of the game working.<br><br>In Stage 2, I helped review the VIPCustomer and CriticCustomer entities. In Stage 3, I was responsible for the Host, ComboMeal, and Happy Hour features. The Host seats the earliest waiting customer at the first free table, once per turn, while leaving the Waiter free to take an action. ComboMeal combines two ordinary menu items, adds their preparation units and charges 90% of their combined price. Happy Hour can be activated once for two turns; it restores up to 15% satisfaction to waiting or unserved customers and discounts orders taken while active by 20%. I also helped verify the integrated game and prepare gameplay and test instructions.<br><br>I also helped improve the documentation, including the README file and this report, to make it easier to understand. |
| Knowledge or skill gained | Building the Stage 1 entities helped me understand how a game's classes share information and how changes to one entity affect the rest of the program. Reviewing the Stage 2 customers strengthened my understanding of method overriding and customer-specific behaviour. In Stage 3, I gained experience using composition to create combo meals, managing a feature that is active for a limited number of turns, and applying discounts at the correct time. Testing complete games also taught me how to use a fixed random seed to reproduce results and trace problems through the turn-by-turn output.<br><br>On top of this, I also learned to use Git commands more effectively in a group project. I practised checking my changes with git status, switching branches, committing my work, and pushing it to GitHub. I gained a clearer understanding of how branches and pull requests help teammates review and combine their work without immediately changing the main branch. |
| Course concept applied | I applied object-oriented programming across the three stages. The entity classes use encapsulation to keep each object responsible for its own state and rules. The customer subclasses demonstrate inheritance and polymorphism: the game can interact with customers through common methods while VIP and Critic customers behave differently. ComboMeal demonstrates composition because it contains two menu items. The Host and Happy Hour features also show separation of responsibilities: the Host manages seating, while Happy Hour manages its activation period and discount calculation. |
| Hardest challenge | My hardest challenge was making sure the Stage 3 features worked correctly together with the existing game flow. Automatic Host seating changes which actions the Waiter can take. Combo meals require the correct price and preparation time. Happy Hour must restore satisfaction during its active turns and discount only orders taken during that period. These rules also affect whether customers leave, pay, or contribute to the victory targets, so I needed to check the events throughout a game rather than rely only on its final result |

| Prompt | Member-specific response |
| --- | --- |
| Attempts and resolution | I reviewed the relevant entities and Stage 3 rules, then compiled and ran the integrated game. I used seed 42 to repeat the same game sequence and checked when the Host seated customers, when combo orders were prepared and served, and whether Happy Hour discounts remained fixed when customers paid later. In one successful test with Happy Hour activated on turn 4, the game finished with 5 paid customers, $90.25 revenue, and 82.20 average satisfaction. I also checked a defeat run to confirm that unsuccessful play produced the expected outcome. From this, I learned to verify both individual feature rules and their effects on the complete game. |

# 9. AI Declaration

Appendix I: Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, Ong Yew Sien, Keane (student name), kong042@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Zhang Jie
4. Title of Assignment/Project Submission: Restaurant Rush Game

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐ And/or

iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.



Ong Yew Sien, Keane 12/11/2026 (Student Name & Signature Date)

Appendix I: Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, Yeo Wei Xuan (student name), yeow0094@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming
2. Course Code: SC2002
3. Instructor: Prof. Zhang Jie
4. Title of Assignment/Project Submission: Restaurant Rush Game

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

I. Used GAI as permitted to assist in generating key ideas only. ☑
II. Used GAI as permitted to assist in generating a first text only. ☐ And/or
III. Used GAI to refine syntax and grammar for correct language submission only. ☐
IV. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ☐

I also declare that I have :

A. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
B. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
C. Apart from the foregoing notices, the submission is wholly my own work.



YEO WEI XUAN 12/11/2026

Student Name & Signature Date

Appendix I: Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, CHUA KAI WEN (student name), I250001@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

5. Name of course: Object Oriented Design and Programming
6. Course Code: SC2002
7. Instructor: Prof. Zhang Jie
8. Title of Assignment/Project Submission: Restaurant Rush Game

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

V. Used GAI as permitted to assist in generating key ideas only. ☑

VI. Used GAI as permitted to assist in generating a first text only. ☐ And/or

VII. Used GAI to refine syntax and grammar for correct language submission only. ☐

VIII. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ☐

I also declare that I have :

D. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
E. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
F. Apart from the foregoing notices, the submission is wholly my own work.



__CHUA KAI WEN____ _12/11/2026__

Student Name & Signature Date

Appendix I: Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, LIM KI RIN (student name), KLIM161@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

9. Name of course: Object Oriented Design and Programming
10. Course Code: SC2002
11. Instructor: Prof. Zhang Jie
12. Title of Assignment/Project Submission: Restaurant Rush Game

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

IX. Used GAI as permitted to assist in generating key ideas only. ☑

X. Used GAI as permitted to assist in generating a first text only. ☐ And/or

XI. Used GAI to refine syntax and grammar for correct language submission only. ☐

XII. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project. ☐

I also declare that I have :

G. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

H. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

I. Apart from the foregoing notices, the submission is wholly my own work.



__LIM KI RIN___ _12/11/2026__

Student Name & Signature Date